

OpenAgenet / OAN White Paper: Open Infrastructure for Trusted Agent Interconnection

Jinliang Xu

China Academy of Information and Communications Technology, Beijing, China

xujinliang@caict.ac.cn; jlxuflly@gmail.com

Abstract—OpenAgenet, abbreviated as OAN, is an open infrastructure project for trusted Agent interconnection. It addresses a problem that becomes visible when Agents move from isolated applications into open, multi-operator networks: before an Agent can safely discover, select, and invoke another Agent, it needs a way to verify identity provenance, governance state, discovery authorization, freshness, and pre-connection trust evidence. OAN is designed as a protocol-neutral trust layer. It does not replace Agent interaction protocols, tool protocols, model orchestration frameworks, or application-level workflows. Instead, it provides Root-governed identity admission, Registrar-assisted onboarding, Root-verified package publication, authorization-aware Discovery, and signed trusted invocation. This white paper explains the motivation, architecture, roles, governance model, relationship with MCP, A2A, and ANP, deployment patterns, cooperation model, blockchain-backed authorization bulletin, prototype status, performance profile, and roadmap of OAN.

Open-source project: <https://github.com/OpenAgenet>

Index Terms—OpenAgenet, OAN, Agent Internet, Agent identity, trusted discovery, governance, MCP, A2A, ANP

1. EXECUTIVE SUMMARY

Large language model based Agents are changing from local assistants into network participants. They call tools, consume data, expose capabilities, delegate work, negotiate task context, and cooperate with other Agents. As this shift continues, the central problem is no longer only how one Agent sends a message to another Agent. A prior infrastructure question must be answered: when an Agent is discovered in an open environment, who says that this Agent is the one it claims to be, who accepted its identity, whether the accepted identity is still current, whether the Discovery service is authorized to expose it, and how a relying Agent can validate these facts before interaction.

OAN addresses this pre-connection trust problem. It defines an open infrastructure architecture for Agent digital identity registration, Root-governed acceptance, trusted package distribution, authorization-aware Discovery, and signed Agent-to-Agent invocation. The design is intentionally layered. OAN focuses on identity, lifecycle governance, discoverability, and trust evidence. It leaves task semantics, tool protocols, model orchestration, dialogue strategy, and application workflow to protocols and frameworks that operate above it.

The project is motivated by a practical observation. A future Agent ecosystem will not be operated by one platform. Agents will be developed by enterprises, research institutes, individual developers, public-service systems, industrial platforms, and

vertical-domain operators. They will run behind different gateways, use different runtimes, and support different interaction protocols. If every operator defines its own identity admission, directory, credential, and trust-verification mechanism, the ecosystem fragments quickly. OAN provides a common trust substrate that allows these heterogeneous Agents to become discoverable and verifiable under shared governance rules.

OAN can be summarized through five commitments:

- **Identity before interaction:** an Agent should have a governed digital identity before it becomes visible to other parties.
- **Lifecycle before directory:** Discovery should expose only Root-accepted, current, and policy-admissible identity versions.
- **Authorization before exposure:** a Discovery service should only expose Agent identities within its authorized capability domains.
- **Verification before invocation:** a relying Agent should verify discovery provenance and request signatures before business interaction.
- **Compatibility before replacement:** OAN should complement MCP, A2A, ANP, DID/VC methods, and domain protocols instead of replacing them.

The result is a governance-oriented Agent interconnection layer. It gives different organizations a way to cooperate without forcing them into one Agent runtime, one model vendor, one application framework, or one task protocol.

2. BACKGROUND: FROM LOCAL AGENTS TO OPEN AGENT NETWORKS

Early Agent systems were often embedded inside one application. The application owner controlled the model, prompt, tool list, user interface, permissions, and execution environment. In that setting, identity and discovery were simple because the platform boundary was clear. The application already knew which components it trusted.

Open Agent interconnection changes this assumption. A User Agent may need to find a professional Service Agent operated by another organization. A Service Agent may rely on tools exposed through MCP servers. A workflow platform may hand a task to an A2A-compatible Agent. A public-domain directory may publish Agents that serve public services, industry data, or enterprise operations. Recent multi-agent systems and surveys show the same shift from isolated Agent prototypes toward networked, tool-using, and collaborative

Agent workflows [1], [2], [3], [4]. This open setting creates four infrastructure gaps.

A. Identity Gap

An Agent needs an identity that is more than a display name, endpoint URL, or marketing description. It needs verifiable subject information, verification methods, service endpoints, capability metadata, and lifecycle state. The identity should be signed, versioned, and traceable to a governance decision. Without such a representation, relying parties cannot distinguish an accepted Agent from an imitation, stale copy, or unreviewed endpoint.

B. Governance Gap

Open networks need a way to decide which infrastructure participants can register Agents, which Discovery services can expose which domains, and which Agent identity versions are accepted. Governance is not only a social process. It must be reflected in machine-verifiable records. Otherwise, operators must manually trust directory contents and cannot automate validation.

C. Discovery Gap

Discovery is not merely search. In an open Agent network, Discovery is a controlled exposure function. It should return candidates that match the query and also satisfy identity provenance, freshness, and authorization constraints. A Discovery node that is allowed to expose manufacturing Agents should not automatically expose medical, financial, or government-service Agents simply because it can fetch their metadata.

D. Invocation Gap

Even after discovery, a Service Agent should not accept every incoming request that names a known Agent. The request should be bound to caller identity, target identity, body hash, timestamp, nonce, and proof. This guard is especially important when Agents trigger actions, retrieve data, or initiate downstream tool calls.

OAN is designed to close these gaps while remaining small enough to be implemented by independent operators.

3. WHY OAN, WHY NOW

The Agent ecosystem is entering a phase where interaction protocols are moving faster than trust infrastructure. MCP makes tools and data sources easier for AI applications to access [5]. A2A-style protocols make Agent-to-Agent task exchange more explicit [6]. Agent DID methods and ANP-like efforts explore identity and networking conventions [7]. These developments are necessary, but they leave a common precondition unresolved: before an Agent enters a tool session, task session, or domain workflow, the relying party must know whether the Agent identity is governed, current, discoverable under policy, and verifiable.

Traditional approaches are not sufficient by themselves. API keys identify access relationships but do not provide open discovery or lifecycle provenance. Private directories work inside one platform but do not scale across organizations.

Ordinary service registries expose endpoints but usually do not encode Agent subject control, governance acceptance, or Discovery authorization. DID documents and Verifiable Credentials provide important identifier and credential substrates [8], [9], but they do not by themselves define who may register an Agent into an ecosystem, which Discovery node may expose it, or how a relying Agent should validate pre-connection evidence.

OAN is timely because open Agent cooperation needs a layer that is neither a closed platform account system nor another task protocol. It is a trust infrastructure layer that can sit beside existing protocols and make them safer to use across organizational boundaries.

4. VISION AND SCOPE

The long-term vision of OAN is an open Agent Internet in which Agents can be registered, discovered, verified, and invoked across organizational boundaries under common trust rules. The project does not assume that the entire world will share one governance root. It assumes that each trust domain can operate a Root authority and that future work can define federation among roots. The current scope is therefore a practical first step: build a clear trust domain model, prove the identity lifecycle, support multiple Registrar and Discovery nodes, and expose enough interfaces for partners to integrate existing Agents.

A. What OAN Is

OAN is:

- a lifecycle model for Agent digital identity;
- a governance model for Root, Registrar, Discovery, and CDN roles;
- a package-distribution model for Root-verified identity artifacts;
- an authorization-aware Discovery model based on capability domains;
- a pre-connection verification model for Agent-to-Agent invocation;
- a reference multi-repository implementation and trial-network basis.

B. What OAN Is Not

OAN is not:

- a replacement for MCP, A2A, ANP, or other interaction protocols;
- a model runtime, LLM inference framework, or Agent-planning engine;
- a ranking algorithm or marketplace business model;
- a universal semantic ontology for all Agent capabilities;
- a centralized platform that must observe all Agent business traffic.

This boundary is important. OAN is deliberately positioned below application protocols and above raw cryptographic primitives. It gives other protocols a shared trust context, but it does not prescribe their internal task semantics. This white paper therefore does not freeze wire formats, API schemas,

canonical serialization, or SDK function signatures. Those implementation details belong in the technical yellow paper, protocol specifications, and conformance artifacts. The purpose here is to define the value, roles, trust path, governance vocabulary, and adoption model that make those lower-level specifications meaningful.

5. DESIGN PRINCIPLES

A. Governance and Protocol Should Meet at Verifiable Facts

Real ecosystems require governance. Operators must authorize infrastructure nodes, review participants, define capability domains, handle revocation, and audit misbehavior. At the same time, runtime systems require machine-verifiable facts. OAN connects these two sides through signed records, Root-verified packages, bulletin facts, credentials, document hashes, and signed responses.

B. Discovery Should Be Policy-Aware

Traditional service discovery often focuses on name, address, or metadata matching [10], [11]. OAN treats discovery as an authorization-sensitive function. A candidate is not admissible merely because it exists and matches a keyword. It must be accepted by Root, current, valid under capability-domain policy, and returned by an authorized Discovery node.

C. Compatibility Should Be Designed In

The Agent ecosystem already contains tool protocols, task protocols, DID methods, API gateways, service meshes, and enterprise identity systems. OAN should not require these systems to be discarded. It should provide adapters, metadata mapping, and pre-connection checks that allow existing protocols to benefit from governed identity and discovery.

D. Trust Should Be Distributed by Role

No single operational component should silently inherit all trust. Registrar helps with onboarding but does not decide final acceptance. CDN distributes packages but does not become a trust authority. Discovery queries and indexes data but must verify Root proof and respect authorization. Service Agents execute business logic but should verify callers and provenance. This role separation reduces implicit trust and creates clearer audit boundaries.

E. The Reference System Should Expose Its Bottlenecks

The prototype is not presented as final Internet-scale infrastructure. It is a working reference that reveals which parts of the lifecycle, publication path, synchronization path, and query path need hardening. OAN treats evaluation as both validation and design feedback.

6. SYSTEM ROLES

OAN uses six main roles. Each role can be operated by one organization in a small pilot or by different organizations in a larger trust domain.

A. Root Node

The Root Node is the governance center of one OAN trust domain. It authorizes infrastructure participants, verifies complete Agent identity submissions, anchors accepted versions, manages capability-domain governance, publishes Root-signed records, and creates verified packages for distribution. Root is not expected to handle ordinary Agent-to-Agent business traffic. Its purpose is to establish which identities and infrastructure nodes are admissible.

Root maintains the following categories of state:

- active and inactive Registrar authorization state;
- active and inactive Discovery authorization state;
- accepted Agent identity versions;
- rejected submissions and verification reasons;
- capability-domain governance metadata;
- publication cursors, package metadata, and bulletin facts.

Root is intentionally conservative. It should reject malformed DID documents, invalid registration credentials, unauthorized Registrar submissions, stale or tampered representations, and capability-domain inconsistencies.

In the next-stage governance design, Root is also the official operation entrance for infrastructure lifecycle decisions. It coordinates and validates operator actions, while a blockchain-backed bulletin records authorization events for Registrar, Discovery, and future third-party VC issuer nodes. This keeps Root as the governance and verification service without making Root the only status-query hub for every infrastructure node.

B. Registrar Node

The Registrar Node is the onboarding gateway. It helps Agent operators prepare identity drafts, select capability tags, prove control of DID verification methods, receive registration credentials, and submit complete identity representations to Root. Registrar improves usability while preserving Root as the final trust authority.

Registrar is not a mere form submission service. It binds operator workflow to protocol evidence. In the reference design, a Registrar draft can move through creation, capability tagging, DID-control challenge, credential issuance, and Root submission. The credential issued by Registrar allows Root to verify that the submitted Agent identity passed the expected onboarding path.

C. Discovery Node

The Discovery Node is a controlled query service. It synchronizes Root-verified packages, verifies Root proof, checks bulletin facts, applies capability-domain authorization, builds local indexes, and returns signed discovery responses. It does not decide that an Agent identity is accepted; it relies on Root proof. It also does not expose every package it can fetch. Its own authorization state limits which capability domains it may index and expose.

Discovery may add ranking, search optimization, local indexes, reputation signals, or domain-specific filters, but these should operate after basic OAN admissibility checks.

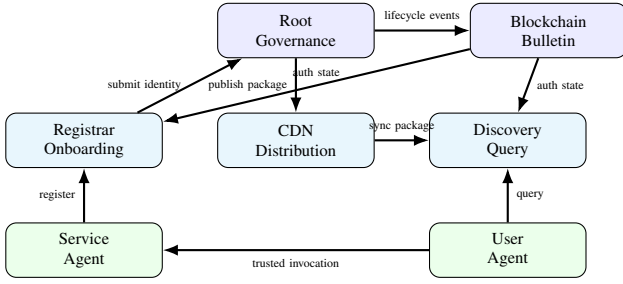


Fig. 1. OAN separates governance, onboarding, distribution, discovery, and Agent interaction.

D. CDN Service

The CDN Service distributes Root-verified packages, manifests, metadata, and documents. It improves availability and reduces load on Root. It is not a trust authority. A Discovery node or relying party should verify hashes, signatures, package metadata, and Root proof rather than trusting the CDN path alone.

E. Service Agent

The Service Agent is a real business Agent that exposes callable capabilities. It may provide domain knowledge, data access, workflow execution, tool access, or task collaboration. In OAN, a Service Agent has an identity document and can be registered, discovered, and invoked under trust controls.

The Service Agent should verify incoming trusted invocation envelopes. It should check caller DID material, credentials, timestamp freshness, nonce uniqueness, target DID binding, request body hash, and signature. These checks allow the Service Agent to reject replay, substitution, wrong-target, and unauthenticated requests before business logic runs.

F. User Agent

The User Agent queries Discovery, verifies signed discovery responses, selects candidates, checks Root-verified package provenance, builds a signed invocation request, and verifies signed Service Agent responses. The User Agent represents the relying side of OAN. It turns Discovery results into verifiable interaction decisions.

7. OAN TRUST ARCHITECTURE

A. Identity Representation

An OAN Agent identity representation is compatible with DID-style concepts. It contains an identifier, verification methods, service endpoints, capability metadata, credential references, OAN metadata, and version-related information. The representation is intended to be complete enough for Root verification and runtime validation. It is not merely a profile card.

The identity representation supports three needs at once. First, it provides human-readable and machine-readable metadata for discovery and integration. Second, it provides cryptographic verification material. Third, it gives Root and Discovery enough information to apply lifecycle and authorization checks.

B. Registration Credential

The registration credential links Registrar review and Agent identity. It binds the Registrar DID, Agent DID, credential type, subject-control evidence, validity, and proof. Root verifies this credential before accepting a submitted identity representation. This creates a separation between review workflow and Root acceptance while preserving verifiability.

C. Root Acceptance

Root acceptance is the point where an Agent identity version becomes authoritative within a trust domain. Root verifies the submitted representation, credential, signed upstream request, Registrar authorization, DID structure, hashes, capability metadata, and policy constraints. If accepted, Root records the version and publishes verifiable evidence. If rejected, the reason should be auditable.

D. Root-Verified Package

A Root-verified package is the distribution artifact consumed by CDN and Discovery. It contains the identity document or references, hashes, metadata, status, publication cursor, Root proof, and capability information. Discovery does not trust a package because it came from a CDN; it verifies the package against Root proof and bulletin facts.

E. Signed Discovery Response

Discovery responses are signed. A relying User Agent can verify that a response came from the Discovery node that produced it, that the response is fresh, and that returned candidates carry provenance sufficient for later validation.

F. Trusted Invocation

Trusted invocation extends OAN from discovery into the first step of interaction. A User Agent signs a request envelope that binds caller DID, target DID, timestamp, nonce, body hash, method, path, and proof. The Service Agent verifies the envelope before executing business logic. OAN does not dictate the complete business protocol after this point, but it establishes a trustworthy entry condition.

8. CAPABILITY DOMAINS AND CONTROLLED DISCOVERY

Capability domains are the governance bridge between identity metadata and discovery exposure. An Agent may describe many capabilities, but a Discovery node should only expose identities in domains for which it is authorized. This is essential for multi-operator deployment. It allows a trust domain to operate domain-specific Discovery services without giving each service global exposure power.

Capability-domain governance has three layers:

- **canonical domains:** a Root-governed capability tree that provides stable top-level and domain-specific categories;
- **authorized Discovery domains:** the subset of the tree that a Discovery node may expose;
- **local refinement tags:** custom tags, descriptions, indexes, or ranking signals used after authorization checks.

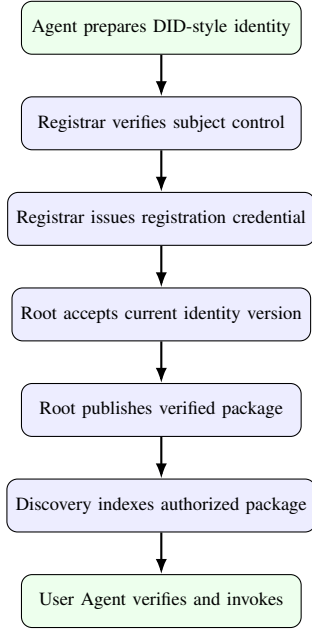


Fig. 2. OAN trust flow from identity preparation to pre-connection invocation.

The key principle is that custom tags cannot expand governance authority. A Discovery node may use custom metadata to improve search quality inside an authorized domain, but it should not use custom tags to expose identities outside its authorized subtree.

9. RELATIONSHIP WITH MCP, A2A, AND ANP

OAN is most useful when understood as a layer around existing protocols rather than a competitor to them.

A. MCP

The Model Context Protocol focuses on connecting AI applications with tools and data sources [5]. It gives clients a structured way to discover and call tools provided by MCP servers. OAN can complement MCP by providing identity and trust context before a tool server or client is used. For example, an Agent can use OAN to verify that an MCP server endpoint belongs to a Root-accepted identity, that the endpoint is current, and that the peer identity has appropriate credentials. MCP remains responsible for tool protocol details.

B. A2A

Agent-to-Agent protocols focus on task exchange, capability description, collaboration state, and message flow between Agents [6]. OAN can provide trusted identity onboarding, discovery, and pre-connection validation for A2A-compatible Agents. In a layered deployment, an Agent might first use OAN to verify a discovered peer and then open an A2A session for task negotiation.

C. ANP and Agent DID Methods

ANP-like work and Agent-oriented DID methods explore identity, networking, and interoperability for Agents [7]. OAN

TABLE I
OAN COMPARED WITH ADJACENT PROTOCOL LAYERS

Layer	Main concern	OAN relationship
MCP	tool and data access	adds identity trust
A2A	Agent task exchange	adds discovery trust
ANP	Agent networking	compatible identity layer
DID/VC	identifier and credential substrate	reused by OAN
Domain APIs	business semantics	can run above OAN

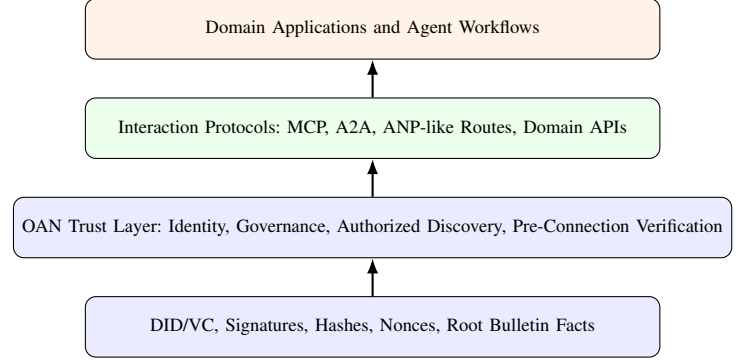


Fig. 3. OAN is positioned as a trust layer below Agent interaction protocols and above cryptographic primitives.

is aligned with DID/VC-style identity and can interoperate with Agent DID methods. Its distinctive focus is the complete governed lifecycle: Registrar authorization, Root acceptance, package publication, capability-scoped Discovery, and signed invocation.

D. Layering Summary

The relationship can be expressed as follows: OAN answers whether an Agent identity is admissible and discoverable; MCP answers how tools are exposed and called; A2A answers how Agents exchange tasks; ANP and DID methods can provide identity and networking conventions; domain protocols answer how specific industries structure their data and workflows.

This layering is the core positioning of OAN. It lets a partner keep an MCP server, an A2A-capable Agent, an ANP-like route structure, or a domain API while adding a common trust path before discovery and invocation. OAN should therefore be evaluated not as a rival to those protocols, but as the layer that can make cross-protocol Agent discovery and first contact governable.

10. OPERATIONAL DEPLOYMENT PATTERNS

A. Single-Operator Trial Network

The simplest deployment places Root, Registrar, Discovery, CDN, Service Agent, and User Agent under one operator. This pattern is useful for development, testing, demonstration, and early governance experiments. It validates lifecycle correctness without introducing cross-organization operations too early.

B. Multi-Registrar Onboarding

As the ecosystem grows, multiple Registrars can onboard Agents for different communities or domains. Root remains the final acceptance authority. This pattern allows domain operators to perform onboarding work while keeping a consistent trust record.

C. Domain-Specific Discovery

Different Discovery nodes can serve different capability domains. For example, one Discovery node may focus on industrial Agents, another on research Agents, and another on public-service Agents. Each Discovery node receives only the authorization it needs.

D. Partner Integration

A partner organization can keep its existing Agent runtime and protocol stack while adding OAN registration, discovery, and verification. It may expose an OAN-compatible identity document, register through a Registrar, synchronize with an authorized Discovery node, and add verification before MCP or A2A interaction.

E. Production Deployment

A production deployment should add database hardening, key management, observability, rate limiting, replay protection storage, backup, monitoring, deployment automation, and conformance tests. OAN's current reference implementation provides the architecture and protocol path; production operation requires ordinary platform engineering around it.

11. GOVERNANCE MODEL

OAN governance is divided between policy decisions and protocol evidence. Policy decisions include who may operate a Registrar, which domains a Discovery node may expose, how capability domains are maintained, what evidence is required for registration, and how revocation is handled. Protocol evidence turns these decisions into signed, verifiable facts.

A. Infrastructure Authorization

Root authorizes infrastructure nodes. A Registrar without Root authorization should not be able to submit accepted Agent identities. A Discovery node without Root authorization should not be trusted as an ecosystem Discovery service. The authorization state should be current, auditable, and visible to relying parties.

B. Blockchain-Backed Bulletin

For infrastructure participants, OAN can publish lifecycle authorization changes through a blockchain-backed bulletin. The bulletin records and emits events for Registrar, Discovery, and future third-party VC issuer lifecycle changes, including authorization, suspension, recovery, revocation, and Discovery-domain updates. Infrastructure nodes subscribe to or poll these events and maintain local authorization caches instead of repeatedly asking the Root service for current status.

This bulletin is not a storage layer for Agent identities. It does not store, parse, validate, or issue DID Documents,

Verifiable Credentials, Agent metadata, or capability descriptions. These remain OAN protocol and service responsibilities. The bulletin may carry external references such as subject identifiers and hashes of off-chain policy material, but its core function is participant lifecycle management.

The initial operation model keeps proposal creation, voting, and lifecycle actions under official governance tooling. Registrar, Discovery, and VC issuer service nodes are event consumers by default. As the ecosystem matures, governance operation clients can be opened to more participating organizations without changing ordinary Agent or service-node integration.

C. Agent Admission

Agent admission involves both operator workflow and protocol checks. A Registrar may help review metadata and verify subject control. Root then verifies the complete representation and supporting evidence. Admission should not depend only on a website account, email address, or human-readable description.

D. Versioning and Revocation

Agents change. Endpoints move, keys rotate, capabilities expand or shrink, and operators may be revoked. OAN treats identity as versioned state. A current version should supersede older accepted versions, and revoked or inactive versions should not be discoverable after synchronization.

E. Auditability

Operators should be able to answer who submitted an identity, which Registrar issued the credential, which Root decision accepted or rejected it, which package was published, which Discovery node indexed it, and why a request was accepted or rejected. This audit path is central to cross-organization trust.

12. SECURITY AND RISK POSTURE

OAN is not a universal security solution. It is closer to zero-trust and workload-identity thinking than to a model-safety framework [12], [13]. It does not guarantee that an Agent's business behavior is correct, truthful, safe, or high quality. It addresses the infrastructure trust questions around identity, governance, discovery, and pre-connection verification.

A. Threats Addressed

The architecture is designed to reduce several risks:

- identity imitation through unsigned or unaccepted metadata;
- stale endpoint use after identity update or revocation;
- unauthorized Registrar submission;
- Discovery exposure outside authorized capability domains;
- tampering with distributed packages;
- replay or wrong-target invocation requests;
- blind trust in directory output without provenance verification.

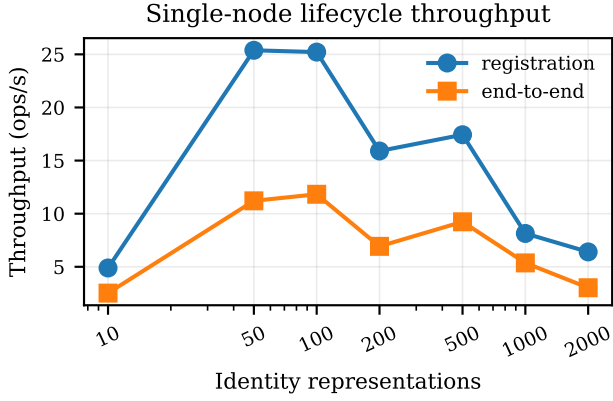


Fig. 4. Single-node lifecycle throughput across increasing identity scales. Registration throughput and end-to-end throughput remain measurable through 2000 identity representations, while the larger scales reveal where production queueing and indexing work should focus.

B. Threats Not Fully Addressed

OAN does not by itself solve prompt injection, model hallucination, malicious tool behavior, data leakage inside business logic, semantic misrepresentation of capabilities, or economic reputation. These require additional application, model, policy, and operational controls. OAN makes it easier to bind those controls to verifiable identities, but it does not replace them.

13. PROTOTYPE AND EVALUATION STATUS

The current OAN prototype has been evaluated through lifecycle correctness, negative verification, authorization-aware Discovery, internal degraded baselines, and scalability characterization. The reported single-node experiments cover 10, 50, 100, 200, 500, 1000, and 2000 identity representations. Logical multi-node experiments cover three Registrar nodes, two Discovery nodes, one Root node, and one CDN node, with per-Registrar scales from 10 to 400, reaching 1200 total identity representations.

The prototype remained operational in the tested scales and returned complete authorized Discovery results in the reported runs. The main bottlenecks appear in large-result query processing, publication, synchronization, and batch submission under high load. These results should be interpreted as feasibility and bottleneck evidence, not as a final production-capacity claim. They also should not be read as a complete production benchmark suite. A production deployment still needs independent measurements for high availability, geographically distributed nodes, long-running synchronization, large tenant populations, and operator-specific storage and queue choices.

Figures 4–6 summarize the main empirical evidence for the white paper. The results support three practical claims: the lifecycle can be executed end to end, authorized Discovery remains complete in the reported runs, and the role-separated design can operate across multiple logical Registrars and Discovery nodes. The same results also identify engineering work

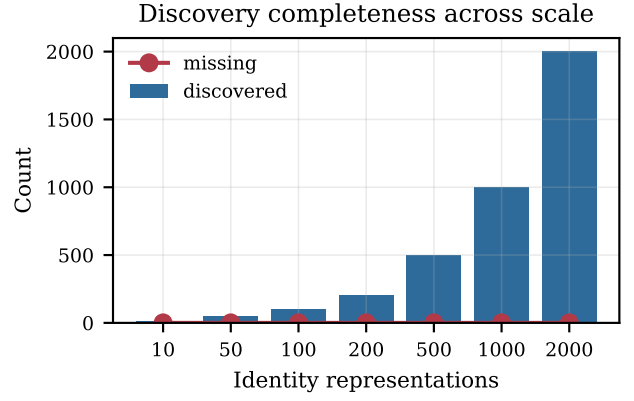


Fig. 5. Discovery completeness in the single-node scalability experiment. Every tested identity representation was discoverable after synchronization, with zero missing results across scales from 10 to 2000.

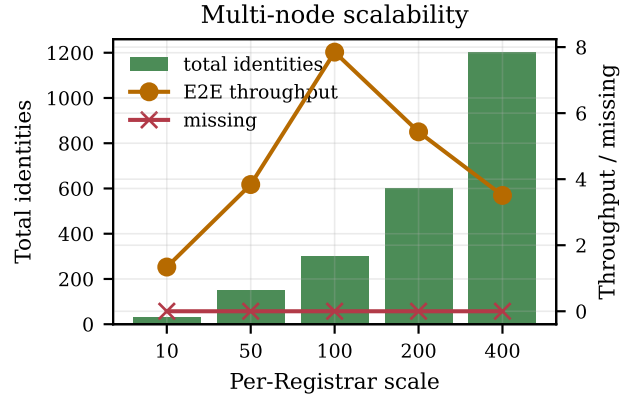


Fig. 6. Logical multi-node scalability with three Registrar nodes and two Discovery nodes. The largest run accepted 1200 total identity representations and both Discovery nodes returned the complete authorized result set.

for future deployments, especially pagination for large result sets, batch tuning, and stronger production storage backends.

14. HOW TO INTERPRET THE PROTOTYPE RESULTS

The current prototype results are important because they validate the coupling among lifecycle governance, Discovery authorization, and trusted invocation. They should be read with the correct expectations. OAN is not claiming that a single reference deployment has already reached global Internet scale. It is claiming that the core trust workflow can be executed end to end, that invalid or unauthorized inputs can be rejected in controlled experiments, and that the main performance bottlenecks are visible enough to guide hardening.

A. Lifecycle Correctness

Lifecycle correctness means that the same Agent identity representation moves through preparation, credentialing, Root acceptance, package publication, Discovery synchronization, query, and invocation without losing the identity binding. This

TABLE II
REPRESENTATIVE EVALUATION COVERAGE

Experiment type	Purpose	Coverage
Lifecycle	verify normal path	registration to query
Negative cases	reject invalid inputs	credentials, proof, scope
Discovery auth	test domain filtering	in-scope and out-of-scope
Ablation	isolate mechanisms	degraded internal baselines
Scalability	find bottlenecks	up to 2000 identities
Multi-node	test role relations	3 Registrar, 2 Discovery

is the most fundamental requirement. A fast system that cannot preserve this binding is not useful as trust infrastructure.

B. Negative Verification

Negative verification demonstrates that the system rejects malformed or unauthorized inputs. These tests are as important as successful-path tests because trust infrastructure is judged by what it refuses to accept. Invalid credentials, wrong subjects, unauthorized infrastructure identities, stale versions, and out-of-scope Discovery cases should fail deterministically.

C. Authorization-Aware Discovery

Authorization-aware Discovery demonstrates that query matching is not enough. The returned candidates must also satisfy Root acceptance, current-version state, and Discovery-domain authorization. This result supports OAN’s central claim that Discovery in an Agent Internet should be a governed exposure function, not only a search box.

D. Scalability Characterization

The scalability results identify where engineering effort should go next. Large query result sets, publication batching, synchronization, and Registrar submission pressure are natural bottlenecks. This is useful because the next round of engineering can focus on pagination, indexing, queue tuning, backpressure, and database backend evolution rather than guessing.

15. STANDARDIZATION CONSIDERATIONS

OAN is not yet a formal standard, but it is designed to produce standardizable surfaces. A standardization path should be careful: it should standardize stable trust semantics before freezing every implementation detail.

A. Candidate Standard Surfaces

Candidate surfaces include Agent identity representation profiles, registration credential claims, signed upstream envelopes, Root-verified package metadata, Discovery response proof format, trusted invocation envelope, capability-domain authorization semantics, and conformance test vectors. These surfaces are useful because they are observable at interoperability boundaries.

B. What Should Remain Flexible

Operator review process, user interface, database backend, local indexing strategy, ranking algorithm, business workflow, and domain-specific Agent protocols should remain flexible. Standardizing these too early would reduce adoption. OAN should standardize the trust path and let ecosystem participants innovate above it.

C. Conformance Before Branding

A project should not be considered OAN-compatible merely because it uses the name or exposes similar endpoints. It should pass conformance tests for object canonicalization, signature verification, credential proof, Root package verification, Discovery authorization, current-version handling, and replay rejection. This protects the ecosystem from shallow compatibility.

16. IMPLEMENTATION GUIDANCE

A. Start with the Trust Path

New implementers should begin with the trust path rather than UI or ranking. The minimal useful path is: create identity representation, prove DID control, issue registration credential, submit to Root, accept and package, synchronize Discovery, query, verify response, and perform signed invocation. Once this path works, local usability and search quality can improve safely.

B. Keep Business Traffic Out of Root

Root should not become a universal business proxy. It should govern identity, infrastructure authorization, and package publication. Business traffic between Agents should be handled by Agent protocols and application systems, with OAN verification used at connection boundaries.

C. Treat Custom Tags Carefully

Custom tags are useful for search and domain adaptation. They are also risky if they accidentally bypass governance. Implementations should treat canonical capability domains as the authorization basis and custom tags as refinement only.

D. Design for Failure

Registrars may be unavailable, Root publication may lag, CDN synchronization may fail, Discovery may be stale, and Service Agents may reject invocation envelopes. A production implementation should surface these states clearly. Operators need to know whether a missing candidate is caused by registration failure, Root rejection, publication delay, Discovery authorization, query mismatch, or runtime verification failure.

17. USE CASES

A. Trusted Public-Service Agent Directory

A public-service operator may need to expose Agents for policy consultation, service navigation, document preparation, or process guidance. OAN allows such Agents to be registered, governed, and discovered under a shared trust domain. Users and platforms can verify that a listed Agent is Root-accepted and current.

B. Industrial Agent Cooperation

Industrial Agents may operate across manufacturing, logistics, quality control, energy management, and supply-chain workflows. Capability-domain Discovery can separate domains while allowing authorized cross-domain cooperation. Existing industrial protocols can remain in place while OAN provides identity and discovery trust.

C. Research and Testbed Collaboration

Universities, laboratories, and standards groups can use OAN as a testbed for Agent identity, discovery, governance, and interoperability. The role separation makes it possible to test multiple Registrar and Discovery operators without rewriting the whole stack.

D. Enterprise Agent Gateway

An enterprise can register internal or partner-facing Agents through a Registrar, expose selected capabilities through a Discovery node, and require OAN verification before external Agents access internal workflows. This pattern can coexist with enterprise IAM, API gateways, and data-governance systems.

18. COOPERATION WITH OTHER ORGANIZATIONS

OAN is useful for cooperation because it does not require partners to abandon their current systems. A partner can keep its own Agent framework, MCP tools, A2A task logic, enterprise workflow engine, or domain API while integrating OAN for identity and discovery.

Practical cooperation can begin with a small set of tasks:

- map the partner's Agent capabilities to OAN capability domains;
- create OAN-compatible identity documents for selected Agents;
- register selected Agents through a trial Registrar;
- expose them through an authorized Discovery node;
- add OAN verification before MCP or A2A sessions;
- define conformance checks and shared test cases.

This lets partners collaborate on the trust layer first, without forcing early agreement on every application protocol detail.

19. STAKEHOLDER VALUE

OAN has different value for different stakeholders. A useful white paper should make these differences explicit because the system is not only a technical component; it is also an ecosystem coordination mechanism.

A. Agent Developers

Agent developers need a way to publish Agents that can be trusted outside their own application. OAN gives them a path to describe identity, endpoints, capabilities, credentials, and verification methods in a format that other systems can inspect. Instead of asking every possible partner to trust a private document or a manually shared endpoint, the developer can point to an accepted OAN identity and a discoverable package.

For developers, OAN also reduces repeated integration work. Once an Agent can produce OAN-compatible identity material and signed responses, it can be made visible through different Discovery nodes and can be invoked by different User Agents. This is especially valuable for small teams that cannot negotiate a custom trust integration with every potential partner.

B. Infrastructure Operators

Infrastructure operators need clear responsibilities. OAN separates Root, Registrar, Discovery, and CDN so that an operator can choose a role without owning the entire ecosystem. A registrar operator can focus on onboarding and evidence collection. A Discovery operator can focus on search, indexing, and domain-specific query quality. A Root operator can focus on governance state, audit, and accepted-version authority. A CDN operator can focus on package availability.

This role separation also allows multiple organizations to participate in a pilot network without collapsing into one centralized service provider.

C. Enterprise Users

Enterprise users usually need three assurances before they connect external Agents to internal workflows. First, the external Agent must be identifiable. Second, its discoverability should be governed. Third, invocation should carry verifiable caller and target context. OAN provides these assurances without forcing the enterprise to expose internal business systems through OAN itself. The enterprise can keep its existing access-control, auditing, and data governance mechanisms while using OAN as a pre-connection trust layer.

D. Public-Service and Industry Platforms

Public-service and industry platforms often require transparent governance and traceability. OAN's Root acceptance records, Registrar credential path, and Discovery authorization model provide a concrete basis for explaining why an Agent appears in a directory. This is important when Agents represent public service functions, regulated-domain guidance, or industrial workflows where uncontrolled exposure can create real operational risk.

E. Standards and Research Communities

OAN provides a concrete implementation target for discussions about Agent identity, Agent discovery, and Agent-to-Agent trust. It is not merely a conceptual diagram. Its multi-repository implementation, tests, examples, and experimental results can support standards discussion, interoperability tests, and academic evaluation.

20. GOVERNANCE OPERATIONS

The OAN governance model should be practical enough for operators. It should not stop at saying that Root is trusted. It should define the kinds of operational decisions that Root and ecosystem maintainers must perform.

A. Registrar Authorization Lifecycle

A Registrar authorization lifecycle includes application, review, activation, monitoring, suspension, and revocation. During application, an operator describes its intended scope, security posture, key management plan, and operational contact. During activation, the governance operator records the Registrar DID and authorization state in the bulletin. During monitoring, Root and ecosystem maintainers may inspect submission quality, rejection patterns, abuse reports, or operational health. Suspension and revocation should be reflected in Root state so that future submissions are rejected.

B. Discovery Authorization Lifecycle

Discovery authorization is domain-sensitive. A Discovery node should receive a defined capability-domain scope. That scope can be expanded, reduced, paused, or revoked. Because Discovery controls exposure, its authorization should be treated as a public trust fact. A blockchain-backed bulletin allows Discovery authorization changes to be sensed by infrastructure nodes through events. A relying Agent should be able to know whether a Discovery node is currently authorized and which domains it may expose through verified Discovery evidence.

C. Capability Tree Governance

The capability tree is not only a technical index. It is a governance artifact. Adding, renaming, splitting, or merging domains can affect which Agents become visible through which Discovery nodes. OAN therefore treats capability tree changes as governance operations. A production ecosystem should keep versions, change records, migration guidance, and compatibility rules for capability domains.

D. Incident Response

An Agent may be compromised, a Registrar may issue invalid credentials, a Discovery node may expose out-of-scope identities, or a package distribution path may be attacked. OAN should support incident response through revocation, state updates, publication of new Root facts, package re-synchronization, Discovery re-indexing, and relying-party verification. The goal is not to prevent every incident, but to make incidents observable and recoverable.

21. DATA AND EVIDENCE MODEL

OAN separates several kinds of data that are often mixed together in simpler directory systems.

A. Public Identity Data

Public identity data includes Agent DID, verification methods, service endpoints, capability tags, public metadata, and Root-verified package references. This data is meant to be used for discovery and verification. It should be sufficient for a relying party to decide whether a candidate is a valid OAN identity and how to approach it.

B. Governance Evidence

Governance evidence may include operator applications, review notes, enterprise approval documents, or offline contact verification. This evidence may be important for governance, but it does not always belong in public packages. OAN therefore allows the protocol-facing trust path to rely on signed credentials and Root records while leaving private evidence in controlled operational systems.

C. Runtime Invocation Evidence

Runtime invocation evidence includes signed request envelopes, nonces, timestamps, selected-service provenance, and signed responses. This evidence is used to verify a specific interaction attempt. It is different from identity admission evidence. Keeping the two separate helps avoid requiring Root to observe every business request.

D. Audit Evidence

Audit evidence links the lifecycle together: which Registrar submitted a representation, which credential was used, which Root record accepted it, which package was published, which Discovery node indexed it, and which response was returned. OAN's architecture is designed so that this chain can be reconstructed without relying on informal claims.

22. INTEROPERABILITY STRATEGY

OAN's interoperability strategy has four layers.

A. Object Compatibility

The first layer is object compatibility. DID-style documents, credential-like claims, signed envelopes, Root packages, and Discovery responses should have stable schemas and canonical verification rules. SDKs should help developers produce and consume these objects without hand-implementing every cryptographic detail.

B. Protocol Adapter Compatibility

The second layer is adapter compatibility. MCP, A2A, ANP-like protocols, REST APIs, and domain protocols can use OAN verification before opening a session or executing a request. The adapter should translate OAN trust context into the local protocol's authentication, authorization, or metadata model.

C. Operational Compatibility

The third layer is operational compatibility. Operators need deployment, configuration, logging, and monitoring conventions that make independent implementations comparable. Without operational compatibility, a protocol can look interoperable on paper while failing in real deployments.

D. Governance Compatibility

The fourth layer is governance compatibility. Different organizations may use different review processes, but they need shared meanings for accepted, revoked, authorized, current, and discoverable states. OAN's Root records and capability-domain model provide a basis for this shared vocabulary.

23. ADOPTION PATH

OAN adoption can be staged. A staged path lowers integration risk and allows partners to choose a depth of participation. The stages are not mandatory in a fixed order. A partner may begin as an SDK consumer, operate a Discovery node for one capability domain, run a Registrar for a governed community, or build an adapter that brings OAN verification into an existing MCP, A2A, ANP-like, or domain-specific protocol. This flexibility is important for cooperation with organizations that already have operational systems and cannot replace them in one step.

A. Stage 1: Identity Publication

In the first stage, an organization creates OAN-compatible identity documents for a small number of Agents. The goal is to make identity material explicit: DID, verification methods, endpoints, capabilities, and basic metadata.

B. Stage 2: Governed Registration

In the second stage, Agents are registered through Registrar and accepted by Root. The organization learns the evidence path, subject-control proof, and Root package publication workflow.

C. Stage 3: Trusted Discovery

In the third stage, an authorized Discovery node indexes accepted packages and returns signed discovery responses. Partner systems can start using OAN Discovery instead of manually curated endpoint lists.

D. Stage 4: Trusted Invocation

In the fourth stage, User Agents and Service Agents use signed invocation envelopes. The integration now covers the full pre-connection path from identity to first verified request.

E. Stage 5: Protocol Integration

In the fifth stage, OAN verification is embedded into MCP, A2A, ANP-like, or domain-specific protocol flows. At this point, OAN becomes part of routine Agent interconnection rather than a separate demonstration.

24. ECONOMIC AND ECOSYSTEM CONSIDERATIONS

Open infrastructure must avoid unnecessary lock-in. OAN's role separation supports different ecosystem models. A public trust domain may operate one Root with multiple independent Registrars and Discovery nodes. An enterprise may operate a private Root for internal and partner Agents. A research testbed may allow experimental roots and compare policy designs. A standards community may use OAN object models and conformance tests without mandating a single public operator.

The ecosystem should also allow specialization. Some Discovery nodes may optimize for low-latency industrial search. Others may optimize for public service transparency. Some Registrars may specialize in enterprise Agents, while others focus on research or developer communities. OAN gives these roles shared trust semantics while allowing different service models above them.

25. LIMITATIONS

OAN's current design has limitations that should be stated clearly.

A. Single-Root Trust Domain

The current architecture describes one Root authority per trust domain. Multi-root federation is an important next step. Until federation is defined, cross-domain trust requires explicit integration or policy mapping.

B. Semantic Quality

OAN verifies identity and governance state. It does not prove that an Agent is competent, safe in all contexts, or truthful in every response. Domain testing, reputation, certification, runtime monitoring, and policy enforcement remain necessary.

C. Privacy

OAN currently emphasizes verifiable discovery. Some environments need private queries, selective disclosure, or confidential capability matching. These are future extensions rather than solved features.

D. Production Hardening

The reference implementation is suitable for architecture validation and trial integration. Production deployment requires hardening around database backends, key custody, rate limits, monitoring, backup, disaster recovery, and operational security.

26. REPOSITORY AND PROJECT STRUCTURE

The OAN implementation is organized as a multi-repository project because OAN is intended to become an ecosystem stack rather than a single demonstration service. The split reflects protocol, runtime, adapter, deployment, operation, and governance boundaries:

- `oan-protocol-common`: the protocol core, including DID/VC helpers, canonical objects, cryptographic utilities, verified package models, Discovery response models, bulletin models, and test-vector foundations;
- `oan-reference-services`: Root, Registrar, Discovery, and CDN reference services that demonstrate the infrastructure roles;
- `oan-agent-py`: Python Agent SDK and reference Service/User Agents for trusted invocation and Agent-side verification;
- `oan-sdk-ts`: TypeScript SDK for web, Node.js, console, and partner integration scenarios;
- `oan-examples`: executable integration tests, demos, security negative cases, MCP/A2A-oriented examples, and benchmark scenarios;
- `oan-adapters`: protocol adapters that connect OAN trust context to MCP, A2A, ANP-like, and domain-specific protocols;
- `oan-discovery-plugins`: extensible Discovery matching, ranking, semantic search, and domain plugin components;

- `oan-deploy`: deployment profiles, local trial orchestration, containerization, and future production deployment assets;
- `oan-web-console`: operator console for Root, Registrar, Discovery, CDN, and trial-network operations;
- `oan-operator-guides`: operational runbooks, onboarding guides, key-management guidance, and cooperation procedures;
- `oan-trial-network`: public trial-network node records, applications, activation state, and ecosystem metadata;
- `oan-release-tools`: signed release, artifact verification, compatibility statement, and supply-chain tooling;
- `oan-site`: public website and official trial-network entry;
- `oan-design-docs`: architecture, governance rationale, design decisions, white papers, yellow papers, and long-form reference documents.

This split prevents protocol definitions, runtime services, examples, deployment scripts, and governance records from collapsing into one repository. It also helps partners integrate only the components they need.

The multi-repository layout is itself part of the OAN adoption story. A partner that wants to implement protocol objects can start with the common protocol layer. A partner that wants to operate infrastructure can start with reference services and operator guides. A partner that only wants Agent-side trust can use the SDKs. A protocol community can work through adapters without rewriting the Root or Discovery services. This is how OAN can grow as open infrastructure instead of a monolithic application.

27. ROADMAP

A. Short-Term

Near-term work should focus on making the reference system easier to run and evaluate. This includes deployment scripts, stable configuration, reproducible test environments, clearer SDK APIs, conformance cases, and documentation for operators.

B. Medium-Term

Medium-term work should improve production readiness: database abstraction, PostgreSQL support, stronger key management, durable nonce storage, pagination, index optimization, observability, rate limiting, and operational dashboards. Adapters for MCP and A2A should move from placeholder compatibility to practical integration patterns.

C. Long-Term

Long-term work includes multi-root federation, privacy-preserving Discovery, cross-domain capability governance, independent implementation conformance, formal security analysis, and broader standardization engagement. The goal is to make OAN a credible trust layer for open Agent interconnection rather than a single project demonstration.

28. CONCLUSION

OAN provides a missing trust-infrastructure layer for open Agent ecosystems. It does not replace Agent interaction protocols, model frameworks, or business workflows. It answers a different set of questions: who is this Agent, who accepted its identity, whether the identity is current, whether a Discovery service is authorized to expose it, and how a peer can validate these facts before interaction.

By separating Root governance, Registrar onboarding, CDN distribution, Discovery exposure, and Agent-to-Agent invocation, OAN gives heterogeneous operators a practical way to cooperate. It is a project for making Agent interconnection verifiable, governable, and compatible with the protocol diversity that will inevitably exist in the Agent Internet.

ACKNOWLEDGMENT

The author thanks Jian Jin, Xie Jiagui, Li Bingqi, and Zhu Runkai for their support.

REFERENCES

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. K. S. Yau, Z. Lin, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [3] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, and C. Wu, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VtmBAGCN7o>
- [4] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [5] Model Context Protocol Contributors, "Model context protocol specification," <https://modelcontextprotocol.io/specification/>, 2025.
- [6] A2A Protocol Contributors, "Agent2agent protocol specification," <https://a2a-protocol.org/latest/specification/>, 2025.
- [7] Agent Network Protocol Contributors, "Did:wba method specification: Web-based agent decentralized identifier," <https://agentnetworkprotocol.com/en/specs/03-did-wba-method-specification/>, 2025.
- [8] W3C Credentials Community Group, "Decentralized identifiers (dids) v1.0," <https://www.w3.org/TR/did-core/>, 2022.
- [9] W3C Verifiable Credentials Working Group, "Verifiable credentials data model v2.0," <https://www.w3.org/TR/vc-data-model-2.0/>, 2025.
- [10] S. Cheshire and M. Krochmal, "Dns-based service discovery," RFC 6763, 2013.
- [11] B. M. Schwartz, M. Bishop, and E. Nygren, "Service binding and parameter specification via the dns (svcb and https resource records)," RFC 9460, 2023.
- [12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST Special Publication 800-207, 2020.
- [13] Cloud Native Computing Foundation, "The spiffe and spire project for workload identity," <https://spiffe.io/>, 2022.
- [14] J. Xu, "Grail: A deep-granularity hybrid resonance framework for real-time agent discovery via slm-enhanced indexing," *arXiv preprint arXiv:2605.02489*, 2026.
- [15] J. Xu and B. Li, "Darwinnet: An evolutionary network architecture for agent-driven protocol synthesis," *arXiv preprint arXiv:2604.01236*, 2026.
- [16] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 15 174–15 186.

- [17] K. Huang, V. S. Narajala, I. Habler, and A. Sheriff, "Agent name service (ans): A universal directory for secure ai agent discovery and interoperability," *arXiv preprint arXiv:2505.10609*, 2025.
- [18] E. Cui, Y. Cheng, R. She, D. Liu, Z. Liang, M. Guo, T. Li, Q. Wei, W. Xing, and Z. Zhong, "Agentdns: A root domain naming system for llm agents," *arXiv preprint arXiv:2505.22368*, 2025.
- [19] R. Raskar, P. Chari, J. Zinky, M. Lambe, J. J. Grogan, S. Wang, R. Ranjan, R. Singhal, S. Gupta, R. Lincourt, R. Bala, A. Joshi, A. Singh, A. Chopra, D. Stripelis, B. B. S. Kumar, and M. Gorskikh, "Beyond dns: Unlocking the internet of ai agents via the nanda index and verified agentfacts," *arXiv preprint arXiv:2507.14263*, 2025.